Video Games ETL Pipeline (GitHub Repository)

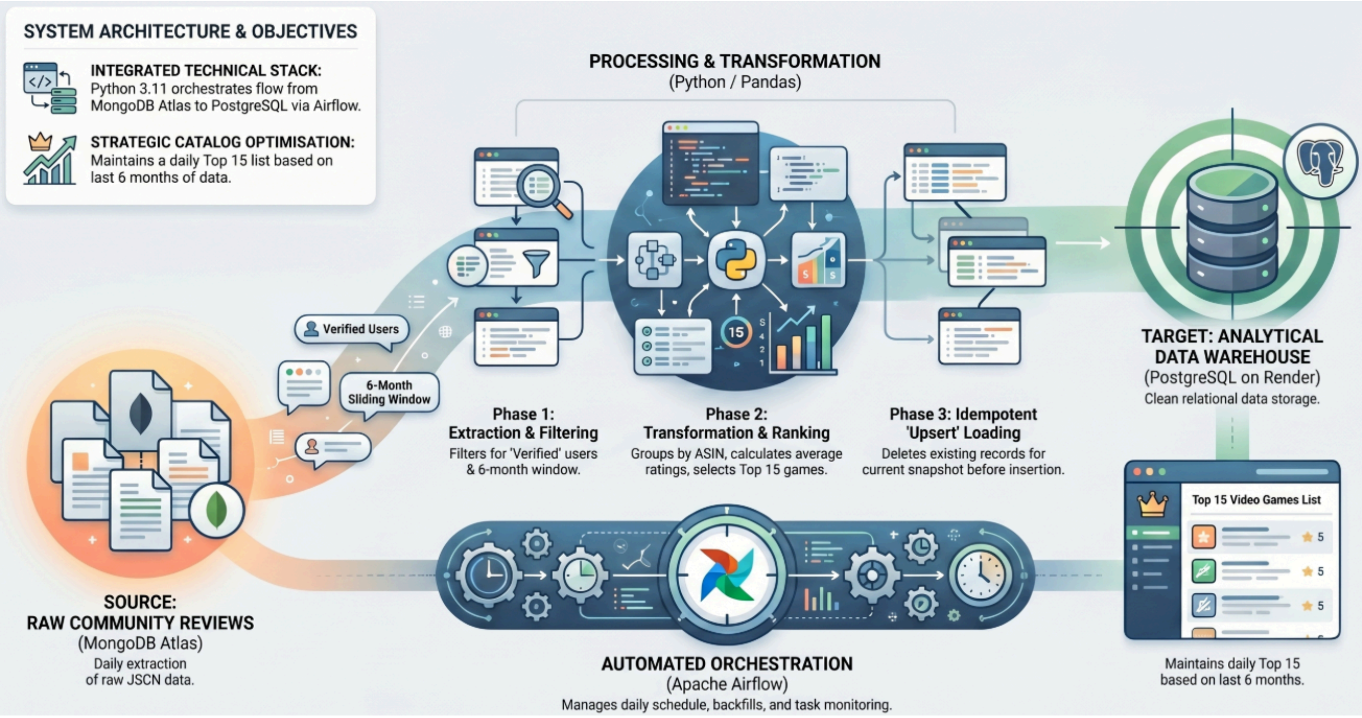
Technical & Operational Documentation

Table of Contents

- 1. General Overview
  - Business Objectives
- 2. Project Specifications
  - 2.1 Business Requirements & Management Rules
  - 2.2 Technical Data Specifications
- 3. Technical Solution & Architecture
  - 3.1 Stack and Tools
  - 3.2 Project Directory Tree
  - 3.3 Architecture Diagram
    - 1. Ingestion and Preparation (Profile: Developer)
    - 2. Manual Pipeline Execution (Profile: Developer)
    - 3. Orchestration and Deployment (Profile: Administrator)
    - 4. Data Consumption (Profile: Analysts & Decision Makers)
  - 3.4 Data Schema
    - Datalake (MongoDB NoSQL)
    - Data Warehouse (SQL)
  - 3.5 Data Referencing
  - 3.6 Traceability Matrix
  - 3.7 Engineering Enhancements
- 4 Deployment Guide (Administrator)
  - Step 1: Cloud Infrastructures Provisioning
  - Step 2: Codebase Cloning and Dependency Setup
  - Step 3: Launching and Initializing Airflow Services
- 5 Developer Guide
  - Direct Local ETL Execution
  - Automated Dataset Seeding
  - Database Maintenance (Date Postdating)
  - Logical Data Flow Sequencing
  - Aggregation Logic Blocks
- 6 Monitoring & Infrastructure Maintenance
  - Orchestration Engine (Airflow):
  - DWH Results:

1. General Overview

This document defines the architecture, configuration, and operation of the automated ETL pipeline. The primary objective is to extract raw customer reviews stored in a NoSQL database daily, identify community trends, and populate a relational Data Warehouse (DWH).



Business Objectives

- **Catalog Optimization**: Feature the top-rated and most appreciated games on the website's homepage and in marketing communication channels (newsletters, social media networks).
- **Data Freshness**: Maintain a day-by-day historical record of the top 15 highest-rated games based exclusively on customer reviews submitted over the last 6 months.

2. Project Specifications

2.1 Business Requirements & Management Rules

- **Rolling Window**: Strict exclusion of any product review submitted more than 6 months prior to the execution date.
- **Top 15**: Daily calculation based on average ratings and overall review volume to extract exactly the top 15 references.
- **Idempotence & Uniqueness**: Zero tolerance for duplicate entries. If the pipeline runs multiple times on the same logical day, existing records for that specific date must be overwritten and replaced (**Upsert / Replace Strategy**).

2.2 Technical Data Specifications

The raw source data originates from a compressed JSON stream ingested into MongoDB Atlas.

- **Source Stream (JSON File)**

  Each entry represents a single customer review containing the following fields:

  - **reviewerID**: Unique identifier of the user.
  - **verified**: Boolean flag indicating if the purchase is verified (ETL filtering criterion).
  - **asin**: Unique product identifier (Amazon Standard Identification Number).
  - **reviewerName**: Name or handle of the author.
  - **vote**: Number of helpful votes received by the review.
  - **style**: Dictionary describing the product format (e.g., "Digital Download").
  - **reviewText**: Text body of the review (dropped during ingestion to optimize cluster storage capacity).
  - **overall**: Numerical rating ranging from 1 to 5.
  - **summary**: Headline or summary title of the review.
  - **unixReviewTime**: Unix Timestamp (used to calculate the dynamic 6-month rolling window).
  - **reviewTime**: Formatted string date (e.g., "05 22, 2024").
  - **image**: A list of URLs pointing to images uploaded by the customer.

- **Datalake (MongoDB NoSQL)**

  The staging collection stores the filtered fields extracted from the raw file required by the target schema. Only the necessary fields from the file need to be fetched.

- **Target DWH (SQL)** The final target relational table features the following columns:

  - Unique game identifier (ASIN)
  - Calculated average rating
  - Total count of users who rated the game
  - Oldest review rating retained in the rolling window
  - Most recent review rating recorded in the rolling window
  - Calculation execution date

3. Technical Solution & Architecture

3.1 Stack and Tools

- **Programming Languages**: Python 3.11, JavaScript
- **Storage Infrastructure**: MongoDB Atlas (Source Database), PostgreSQL on Render (Data Warehouse).
- **Data Processing**: Pandas & MongoDB Aggregation Framework.
- **Orchestration Engine**: Apache Airflow (Local Environment / Server Instance).
- **IDE**: Visual Studio Code
  - **Extensions**:
    - MongoDB for VS Code
    - SQLTools & SQLTools PostgreSQL/Cockroach Driver
    - SQLite Viewer
- **Operating System**: Ubuntu Linux. (The Airflow Scheduler strictly requires a Unix-like OS environment to support task process handling via `os.fork`).

3.2 Project Directory Tree

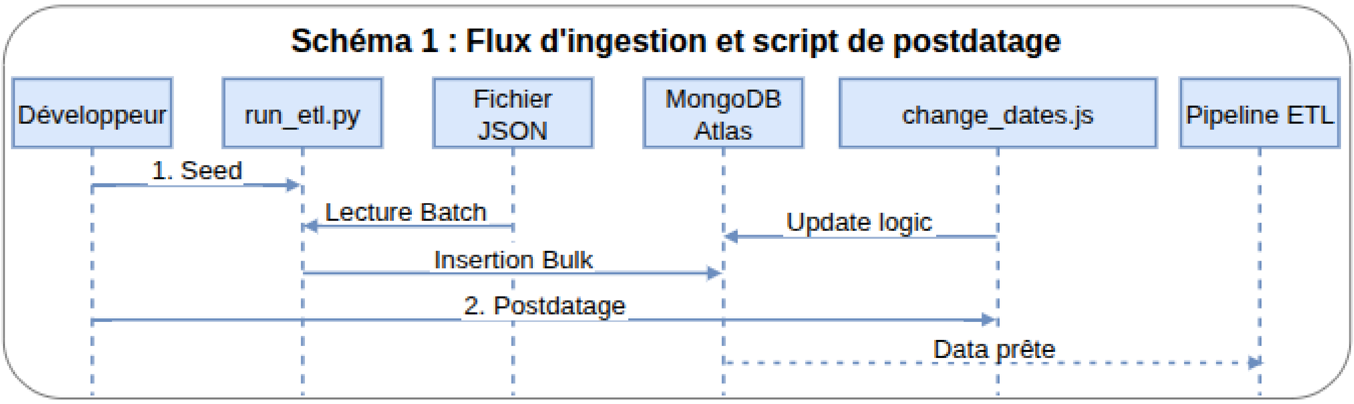
```
BlentDataProject/
├── .venv_airflow/          # Dedicated virtual environment for Airflow
├── .venv_etl/              # Dedicated virtual environment for the ETL script
├── airflow_home/           # Airflow working directory (Logs, local DB)
│   ├── airflow.db          # Orchestrator SQLite database
│   └── dags/               # Airflow DAGs folder
│       └── dag_task_etl.py # Airflow 2.3 DAG definition & integrated documentation
├── doc/                    # Technical and functional specifications
│   └── md/
│       ├── doc_en.md       # Technical Documentation (English)
│       ├── doc_fr.md       # Documentation Technique (Français)
│       └── spec_fr.md      # Initial specifications and requirements
├── queries/                # Maintenance scripts (MongoDB/SQL migrations)
│   ├── datalake/
│   │   └── change_dates.mongodb.js # Date shifting script for MongoDB
│   └── dwh/
│       └── results.csv     # Results from table `daily_snapshot` in DWH DB
├── scripts/
│   └── run_etl.py          # Python script (Extraction, Calculations, Loading)
├── src/                    # Core logic
│   ├── config.py           # Configuration and environment loading
│   └── lib_etl.py          # ETL library and helper functions
├── .env.template           # Secrets template (MongoDB, Postgres)
├── .gitignore              # Exclusions for virtual environments, logs, and .env
├── airflow.env.template    # Path environment variables (PROJECT_ROOT, AIRFLOW_HOME)
├── airflow_run_etl.sh      # Control script (Servers & execution modes)
├── README.md               # Overview and quick-start guide
├── requirements_airflow.txt # Orchestrator dependencies
└── requirements_etl.txt    # ETL script dependencies
```

3.3 Architecture Diagram

This section details data streams and architecture components mapped across different user profiles and business roles.

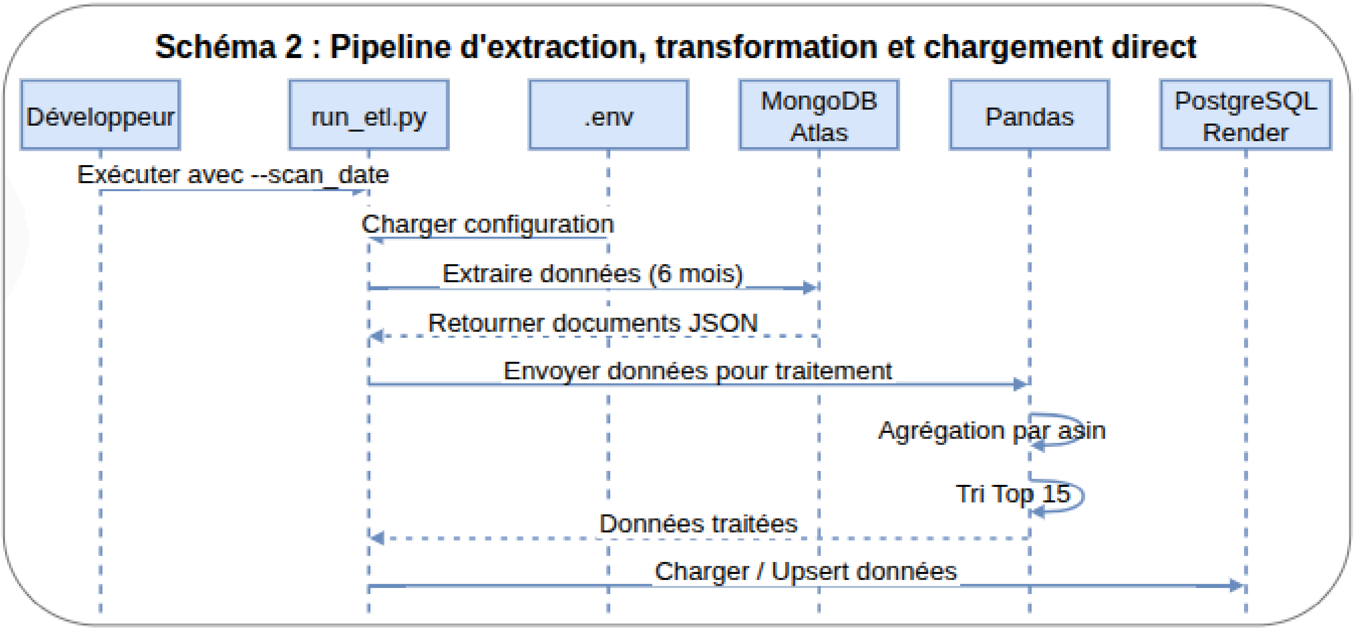
1. Ingestion and Preparation (Profile: Developer)

- **Objective**: Seed the Datalake with workable development datasets.
- **Process**: Loading the source JSON file using the `seed_datalake` function, followed by a date-shifting script to map older documents into a modern 6-month execution window.
- *Ref. Diagram 1: Ingestion workflow and timestamp-shifting script*



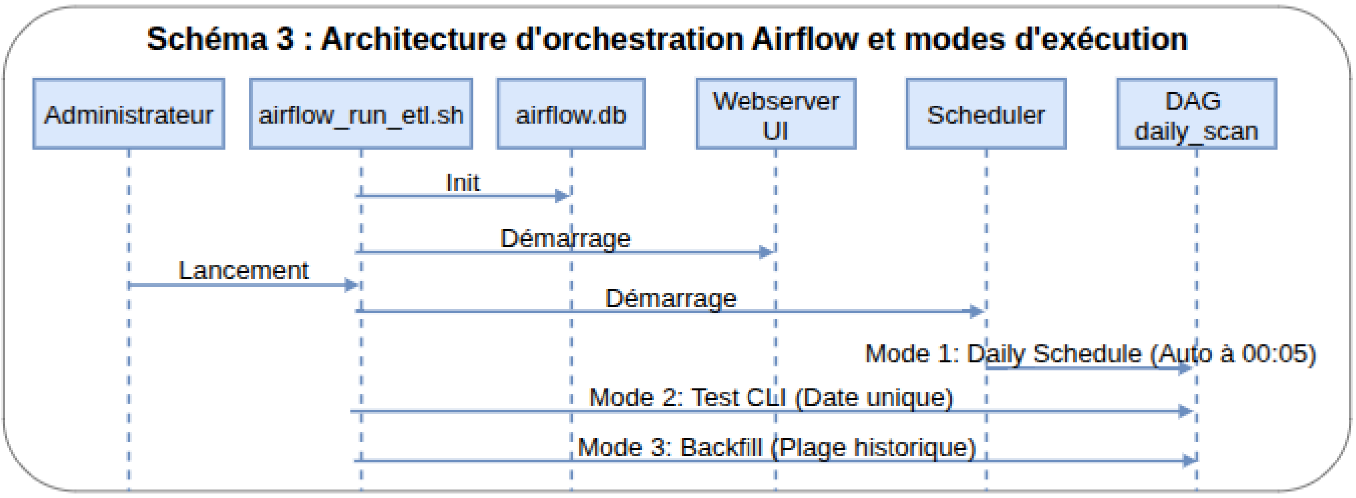
2. Manual Pipeline Execution (Profile: Developer)

- **Objective**: Perform isolated unit testing or execute performance benchmarks.
- **Process**: Triggering the entry-point script `run_etl.py` directly from the CLI using `--scan_date` and `--platform` arguments.
- *Ref. Diagram 2: Direct Extraction, Transformation, and Loading pipeline*



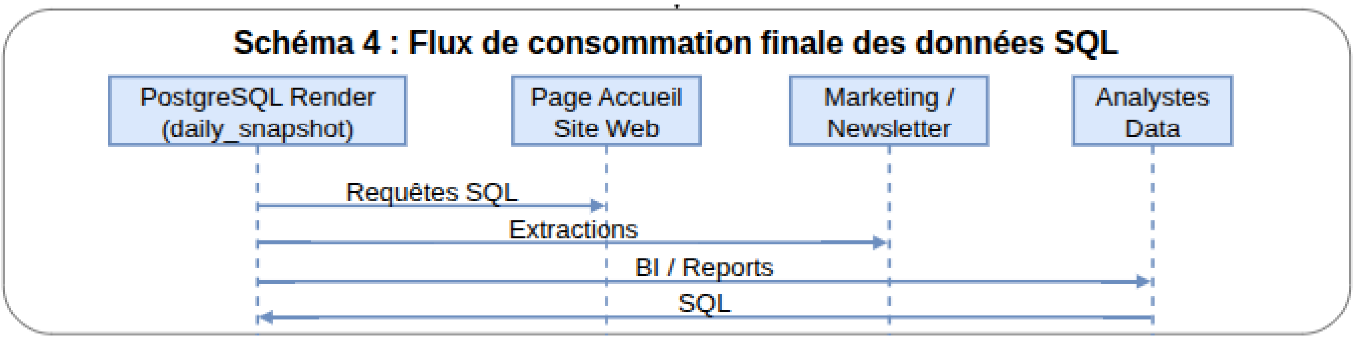
3. Orchestration and Deployment (Profile: Administrator)

- **Objective**: Management of background service infrastructure and production automation.
- **Components**: A control shell script `airflow_run_etl.sh` driving 2 daemon processes (Webserver & Scheduler) paired with an `airflow.db` metadata instance.
- **Operation Modes**: Daily Schedule (Automated), Test (Unit testing), Backfill (Historical catching up).
- *Ref. Diagram 3: Airflow orchestration architecture and execution modes*



4. Data Consumption (Profile: Analysts & Decision Makers)

- **Objective**: Business trend monitoring and decision support analytics.
- **Flow**: Render PostgreSQL DWH ➔ Standard SQL Queries ➔ Analytics Reports / Marketing Newsletters / Web Catalog App.
- *Ref. Diagram 4: Final SQL data consumption workflow*



3.4 Data Schema

Datalake (MongoDB NoSQL)

The collection schema stores data with the following technical fields:

Column	Type	Description
`asin`	str	Unique product identifier (ASIN).
`reviewerID`	str	Unique user identifier.
`overall`	float	Numerical rating from 1 to 5.
`verified`	bool	Boolean flag indicating a verified purchase.
`unixReviewTime`	int	Unix Epoch Timestamp.
`reviewTime`	str	Formatted date text.

Data Warehouse (SQL)

The target relational table `daily_snapshot` enforces the following schema to ensure historical consistency and idempotence:

Column	Type	Description
`product_id`	VARCHAR(50)	Unique game identifier (ASIN).
`snapshot_date`	DATE	Pipeline calculation execution day (Composite Primary Key with product_id).
`nb_reviews`	INTEGER	Number of valid reviews aggregated within the 6-month window.
`average_rating`	NUMERIC(3,2)	Calculated arithmetic mean rating (1.00 to 5.00).
`oldest_rating`	NUMERIC(3,2)	Rating value of the oldest review in the current period.
`newest_rating`	NUMERIC(3,2)	Rating value of the latest review in the current period.

3.5 Data Referencing

In accordance with architectural requirements, no raw data or large files are stored in this Git repository. The repository is strictly reserved for scripts (Python and Shell) and configuration files. To link your GitHub environment to these remote resources, follow this referencing structure:

- **1. Source Storage (Initial Injection)**

  The raw source containing the video game rating data is hosted on an object storage bucket (Amazon S3). It serves as the single point of entry for the Datalake's seeding (initial population) process.

  - Resource Type: Raw data file (JSON compressed in ZIP format)
  - Direct Download Link: games_ratings.zip
  - Dedicated Configuration Variable: SEED_FILE (contains the source file URL for the initialization phase).

- **2. Datalake (NoSQL Collection)**

  The ingestion and intermediate staging layer (Datalake) is deployed on a Cloud cluster to ensure horizontal scalability when receiving JSON streams. DB credentials are provided on request.

  - Hosting Platform: MongoDB Atlas (Database-as-a-Service)
  - Administration Console: MongoDB Atlas Dashboard
  - Database: db_datalake
  - Required Environment Variables (.env):
    - MONGO_URI
    - MONGO_DB
    - MONGO_COLLECTION

- **3. Data Warehouse (SQL Table)**

  The final destination layer, containing the modeled structures ready for business intelligence (BI) analysis, is hosted on a managed relational DB instance. DB credentials are provided on request.

  - Hosting Platform: PostgreSQL on the Render Cloud platform
  - Administration Console: Render Dashboard
  - Database: db_dwh
  - Required Environment Variables (.env):
    - POSTGRES_DSN
    - POSTGRES_TABLE_NAME

3.6 Traceability Matrix

Requirement Segment	Specific Requirement	Script (folder)	Implementation Status & Comments
Source Data	Raw dataset hosted in MongoDB	`lib_etl.py` (src)	Implemented. `connect_mongo` and `extract_and_transform` use `pymongo` to query the source cluster collection.
Data Warehouse	SQL Compliant Infrastructure (PostgreSQL)	`lib_etl.py` (src)	Implemented. Uses `sqlalchemy` and `psycopg2` (via explicit connection string DSN) pointing to the Render PostgreSQL instance.
Data Filtering	Restrict dataset to the last 6 months	`lib_etl.py` (src)	Implemented. `get_timeframe_start` calculates the 6-month delta, and `extract_and_transform` leverages a `$match` operator on `unixReviewTime`.
Aggregation Rules	Extract top 15 highest-rated games	`lib_etl.py` (src)	Implemented. Aggregation pipeline enforces a `$limit: top_n` (defaulting to 15) after sorting records by `average_rating` and `nb_reviews`.
Data Schema Matching	Product ID, Average, Count, Oldest/Newest bound ratings	`lib_etl.py` (src)	Implemented. `init_dwh` initializes the schema with appropriate constraints for `product_id`, `nb_reviews`, `average_rating`, `oldest_rating`, and `newest_rating`.
Idempotence Enforcer	Handle duplicate execution runs cleanly	`lib_etl.py` (src)	Implemented. `upsert_dwh` executes an atomic row `DELETE` for the targeted `snapshot_date` right before triggering the DataFrame `append`.
Core App Logic	Standalone Python execution script	`run_etl.py` (src)	Implemented. The primary CLI entry point script coordinates individual Extract, Transform, and Load phases sequentially.
Automation	Orchestration & Job Scheduling engine	`airflow_run_etl.sh`	Implemented. Control Bash script manages local environment properties, background daemon systems, and explicit backfilling.
Data Integrity	Filter out unverified product reviews	`lib_etl.py` (src)	Implemented. The primary MongoDB server-side aggregation stage explicitly filters records where `verified: True`.

Project Status Assessment Summary:

- **Date Management**: Transitioning legacy sample datasets (dated around 2017) to a modern execution timeline was successfully implemented using the maintenance utility `queries/datalake/change_dates.mongodb.js`. This guarantees that the dynamic 6-month historical calculations yield consistent rows during active production testing.
- **Schema Consistency**: The database creation DDL block within `src/lib_etl.py` configures a composite `PRIMARY KEY (product_id, snapshot_date)`. This fulfills the requirement to eliminate multi-run data pollution while preserving true daily metrics tracking for individual games over time.

3.7 Engineering Enhancements

1. **Performance Optimization**: Offloading computations using the native MongoDB `aggregate` pipeline reduces network overhead and minimizes data transfer into Python.
2. **Pipeline Robustness**: Implementing contextual SQLAlchemy database transactions (`db_dwh.begin()`) protects data warehouse integrity during append sequences.
3. **Security Architecture**: Completely decoupled secret configurations via local `.env` environment isolation.

4 Deployment Guide (Administrator)

Step 1: Cloud Infrastructures Provisioning

1. **MongoDB Atlas (Source Datalake):**
   - Deploy a Shared/Free M0 tier cluster instance.
   - Under **Network Access**, whitelist your runner IP address (or `0.0.0.0/0` during development).
   - Under **Database Access**, create a dedicated application user account with `readWrite` access privileges (required to seed initial testing objects).
   - Retrieve your secure cluster connection URI string (`mongodb+srv://...`).
2. **Render PostgreSQL (Data Warehouse Target):**
   - Provision a new managed **PostgreSQL** cloud instance.
   - Configure database user credentials.
   - Copy down the **External Connection String**.

Step 2: Codebase Cloning and Dependency Setup

```
# 1. Clone the project repository
git clone https://github.com/votre-compte/BlentDataProject.git
cd BlentDataProject

# 2. Configure application secrets
cp .env.template .env
# Edit the newly created .env file with your custom MongoDB and PostgreSQL strings

# 3. Initialize separate virtual environments
python3.11 -m venv .venv_airflow
python3.13 -m venv .venv_etl

# 4. Install requirements packages across environments
source .venv_airflow/bin/activate && pip install -r requirements_airflow.txt && deactivate
source .venv_etl/bin/activate && pip install -r requirements_etl.txt && deactivate
```

Step 3: Launching and Initializing Airflow Services

Execute the primary orchestration control script to spin up background operations:

```
chmod +x airflow_run_etl.sh
./airflow_run_etl.sh
```

This management script automatically orchestrates:

1. The provisioning of the `airflow_home` directory structures and initializing `airflow.db`.
2. The programmatic injection of an `admin` security profile account.
3. The clean initialization of the Airflow **Scheduler** and **Webserver** (Port 8080) as background daemons.

5 Developer Guide

Direct Local ETL Execution

Developers can bypass the Airflow UI layers to run unit tests on the pipeline processing layer directly:

```
source .venv_etl/bin/activate

# Execute the pipeline using the current runtime date context
python scripts/run_etl.py

# Execute the pipeline passing an explicit target date (ISO format string)
python scripts/run_etl.py --scan_date 2024-05-20 --platform Terminal
```

Automated Dataset Seeding

The entry point script `run_etl.py` handles bootstrap actions during its very first initialization sequence:

- **MongoDB Atlas Seeding**: If the destination staging database contains zero documents, the system triggers `seed_datalake` to automatically parse and load items from the packaged JSON file.
- **PostgreSQL DDL Initialization**: The inner utility `init_dwh` automatically provisions the target relational schema layout `daily_snapshot` alongside optimized tracking indices if they do not exist.

Database Maintenance (Date Postdating)

If testing datasets fall outside your 6-month processing calculation scope:

- Open your workspace using Visual Studio Code.
- Access your target server connection through the official MongoDB Extension.
- Open the query scratchpad file located at `queries/datalake/change_dates.mongodb.js`.
- Click the interactive execution **Play** icon to shift timestamps forward.

Logical Data Flow Sequencing

1. **Extract**: Execute a targeted aggregation pipeline matching documents where `unixReviewTime` (Date - 6 months) while verifying that `verified: true`.
2. **Transform**:
    - Sort records chronologically.
    - Group values under distinct `asin` document clusters.
    - Compute arithmetic rating means and isolate initial boundaries (`$first`, `$last`).
    - Order structural output by `average_rating` and `nb_reviews` fields descending.
    - Enforce a processing limit constraint to truncate results down to the top 15 records.
3. **Load**:
    - Open an isolated database transaction window.
    - Clear out prior target tracking entries matching the current operational `snapshot_date`.
    - Perform a high-speed relational bulk insert using the processed data framework.

Aggregation Logic Blocks

The core metric computation engine utilizes the server-side MongoDB pipeline within `extract_and_transform`. This architecture offloads calculations, filtering, and statistical sorting tasks to the hosting database machine.

```
# Extracted pipeline code block definition
pipeline = [
    {"$match": {"unixReviewTime": {"$gte": timeframe_start}, "verified": True}},
    {"$sort": {"unixReviewTime": 1}},
    {"$group": {
        "_id": "$asin",
        "nb_reviews": {"$sum": 1},
        "average_rating": {"$avg": "$overall"},
        "oldest_rating": {"$first": "$overall"},
        "newest_rating": {"$last": "$overall"}
    }},
    {"$sort": {"average_rating": -1, "nb_reviews": -1}},
    {"$limit": 15}
]
```

6 Monitoring & Infrastructure Maintenance

Orchestration Engine (Airflow):

- **Job Triggering & Service CLI Interfaces:**
  - **Spin Up Core Servers**: Run `./airflow_run_etl.sh` without passing any command-line parameters.
  - **Isolated Date Unit Verification**: Run the shell utility passing a target ISO date string parameter (e.g., `./airflow_run_etl.sh 2024-05-20`). This boots an internal `airflow tasks test` routine to validate pipeline logic without recording state mutations in the scheduler history database.
  - **Historical Backfilling**: To process an extensive chronological window on demand, invoke: `./airflow_run_etl.sh <start_date> <end_date>`.
    - *Important Note*: The global absolute lower-bound start constraints are defined via configuration properties within `airflow.env`. Verify that valid source records exist on your MongoDB instances before backfilling target periods.
- **System Health Monitoring:**
  - **Web Management Portal**: Connect to the running Airflow server UI by loading `http://localhost:8080`.
    - **Control Panel Dashboard**: Toggle the switch to activate the target `daily_scan` scheduling graph.

DWH Results:

Run verification scripts directly against your tables to check the computed results for a specific execution run:

```sql
SELECT * FROM public.daily_snapshot
WHERE snapshot_date = <TARGET_DATE YYYY-MM-DD>;
```

- Sample results from table `daily_snapshot` (cf. Results in CSV file):

product_id	nb_reviews	average_rating	oldest_rating	newest_rating	snapshot_date
B01AVLWBY0	13	5.00	5.00	5.00	2026-05-19
B01D014G9Q	13	5.00	5.00	5.00	2026-05-19
B01GWGXHKK	14	5.00	5.00	5.00	2026-05-19
B00000F1GM	14	5.00	5.00	5.00	2026-05-19
B000ERVMI8	15	5.00	5.00	5.00	2026-05-19
B00003OTI3	15	5.00	5.00	5.00	2026-05-19
B01GW8ZA9Y	15	5.00	5.00	5.00	2026-05-19
B00U6DTGP6	15	5.00	5.00	5.00	2026-05-19
B019J6RYCW	17	5.00	5.00	5.00	2026-05-19
B00BAWXD88	18	5.00	5.00	5.00	2026-05-19
B00002STXQ	18	5.00	5.00	5.00	2026-05-19
B014P7QI6I	19	5.00	5.00	5.00	2026-05-19
B00KAED7OC	19	5.00	5.00	5.00	2026-05-19

product_id	nb_reviews	average_rating	oldest_rating	newest_rating	snapshot_date
B00O9GW8VK	25	5.00	5.00	5.00	2026-05-19
B01BHMS88A	30	5.00	5.00	5.00	2026-05-19
B01D014G9Q	13	5.00	5.00	5.00	2026-05-18
B01AVLWBY0	13	5.00	5.00	5.00	2026-05-18
B00000F1GM	14	5.00	5.00	5.00	2026-05-18
B01GWGXHKK	14	5.00	5.00	5.00	2026-05-18
B00U6DTGP6	15	5.00	5.00	5.00	2026-05-18
B000ERVMI8	15	5.00	5.00	5.00	2026-05-18
B01GW8ZA9Y	15	5.00	5.00	5.00	2026-05-18
B00003OTI3	15	5.00	5.00	5.00	2026-05-18
B019J6RYCW	17	5.00	5.00	5.00	2026-05-18
B00BAWXD88	18	5.00	5.00	5.00	2026-05-18
B00002STXQ	18	5.00	5.00	5.00	2026-05-18
B014P7QI6I	19	5.00	5.00	5.00	2026-05-18
B00KAED7OC	19	5.00	5.00	5.00	2026-05-18
B00O9GW8VK	25	5.00	5.00	5.00	2026-05-18
B01BHMS88A	32	5.00	5.00	5.00	2026-05-18
B01AVLWBY0	13	5.00	5.00	5.00	2026-05-17
B01D014G9Q	13	5.00	5.00	5.00	2026-05-17
B01GWGXHKK	14	5.00	5.00	5.00	2026-05-17
B00000F1GM	14	5.00	5.00	5.00	2026-05-17
B01GW8ZA9Y	15	5.00	5.00	5.00	2026-05-17
B000ERVMI8	15	5.00	5.00	5.00	2026-05-17
B00U6DTGP6	15	5.00	5.00	5.00	2026-05-17
B00003OTI3	15	5.00	5.00	5.00	2026-05-17
B019J6RYCW	17	5.00	5.00	5.00	2026-05-17
B00002STXQ	18	5.00	5.00	5.00	2026-05-17
B00BAWXD88	18	5.00	5.00	5.00	2026-05-17
B00KAED7OC	19	5.00	5.00	5.00	2026-05-17
B014P7QI6I	19	5.00	5.00	5.00	2026-05-17
B00O9GW8VK	26	5.00	5.00	5.00	2026-05-17
B01BHMS88A	32	5.00	5.00	5.00	2026-05-17
B01D014G9Q	13	5.00	5.00	5.00	2026-05-16
B01AVLWBY0	13	5.00	5.00	5.00	2026-05-16
B00000F1GM	14	5.00	5.00	5.00	2026-05-16
B01GWGXHKK	14	5.00	5.00	5.00	2026-05-16
B000ERVMI8	15	5.00	5.00	5.00	2026-05-16
B00003OTI3	15	5.00	5.00	5.00	2026-05-16
B00U6DTGP6	15	5.00	5.00	5.00	2026-05-16
B01GW8ZA9Y	15	5.00	5.00	5.00	2026-05-16
B019J6RYCW	17	5.00	5.00	5.00	2026-05-16
B00BAWXD88	18	5.00	5.00	5.00	2026-05-16
B00002STXQ	18	5.00	5.00	5.00	2026-05-16
B00KAED7OC	19	5.00	5.00	5.00	2026-05-16
B014P7QI6I	19	5.00	5.00	5.00	2026-05-16
B00O9GW8VK	26	5.00	5.00	5.00	2026-05-16
B01BHMS88A	32	5.00	5.00	5.00	2026-05-16